Manufacturing Dissent: Verifier Gates for Consistent Limit-Awareness in a Fast-Inference gpt-oss-120b Agent

Tushar Shetty
Independent
tusharshetty61@gmail.com

Abstract

CAR-bench measures whether a tool-using agent knows its own limits: it must abstain when a required tool is missing (Hallucination) and gather information rather than guess when a request is under-specified (Disambiguation) [Kirmayr et al., 2026]. We show that on both task types the failures of gpt-oss-120b are a unanimous completion bias, not high-variance noise. This has a sharp consequence for how Track-2 inference compute should be spent: best-of-N or majority voting over the executor's own action is inert, because there is no cautious minority to harvest. We instead spend the compute on independent, differently-framed verifier gates that run in one parallel burst every step, are themselves voted for stability, and veto the action asymmetrically: abstain when ungrounded, gather when ambiguous. On the held-out test split our agent reaches an average Pass^3 of 0.413 (Base 0.50 / Hallucination 0.42 / Disambiguation 0.32), versus 0.27 for the raw single-pass baseline (deltas +0.14 / +0.07 / +0.21), at a median 2.5 s per decision step. We are deliberate about attribution: of four scaffolding levers we built, three were killed by matched ablations and only the ambiguity gate survives as a genuine contribution. It manufactures disambiguation behaviour from a baseline that produces it zero times on unseen tasks. The Base and Hallucination gains, by contrast, come from the scaffold and greedy decoding, not from any gate.

1 Problem and Thesis

CAR-bench evaluates an in-car assistant across 58 interconnected tools and a set of domain policies, using an LLM-simulated user and multi-turn episodes [Kirmayr et al., 2026]. Beyond ordinary task completion (Base), two task types probe self-awareness. Hallucination tasks remove a tool the goal needs; the correct behaviour is to say so and stop, not to fabricate a plausible workaround. Disambiguation tasks under-specify the request; the correct behaviour is to resolve the gap by consulting policy, stored preferences, or context, and only ask the user as a last resort. The benchmark scores consistency with Pass^k: an episode counts only if all k independent trials pass, so a policy the model obeys two times out of three still fails.

The paper frames the root cause as a completion–compliance tension: RLHF rewards producing a fluent completion, so the model acts or fabricates instead of admitting a limit. It also observes that the capability is usually present but its activation is not stable across trials, which is exactly what Pass^3 punishes, and that more "thinking" does not fix premature action (roughly 90% of GPT-5's Disambiguation failures persist under reasoning effort). The paper's own conclusion is that models lack a "stable activation mechanism" for constraints.

CAR-bench sits in a line of interactive tool-use benchmarks that move past single-turn evaluation, like τ-bench [Yao et al., 2024] and ToolSandbox [Lu et al., 2025], but is the first to score limit-awareness and disambiguation directly. We compete in the Cerebras fast-reasoning track: direct gpt-oss-120b [OpenAI, 2025] inference on the Cerebras wafer-scale engine [Cerebras Systems, 2024], wrapped in an Agent2Agent (A2A) executor harness [Google, 2025]. It is judged on (1) innovative use of inference-time compute, (2) Pass^3 delta over the raw single-pass baseline, and (3) this report. The hard constraint is at most five sequential LLM calls per decision step; parallel calls are free. Our thesis is that the stable activation mechanism the paper asks for is a small ensemble of external verifiers, and that fast inference is precisely what makes running them every step affordable.

Why voting the action cannot work. The instinctive way to spend a $\sim$10$\times$ token budget is best-of-N over the executor. We measured what that buys before building anything. Sampling the raw executor five times on canonical Hallucination and Disambiguation probes returns unanimous verdicts: five fabrications, or five premature actions, at every reasoning effort. When the samples agree, the majority equals the single sample, and voting is a no-op. The cautious answer is not a low-probability sample you can surface with more draws; it

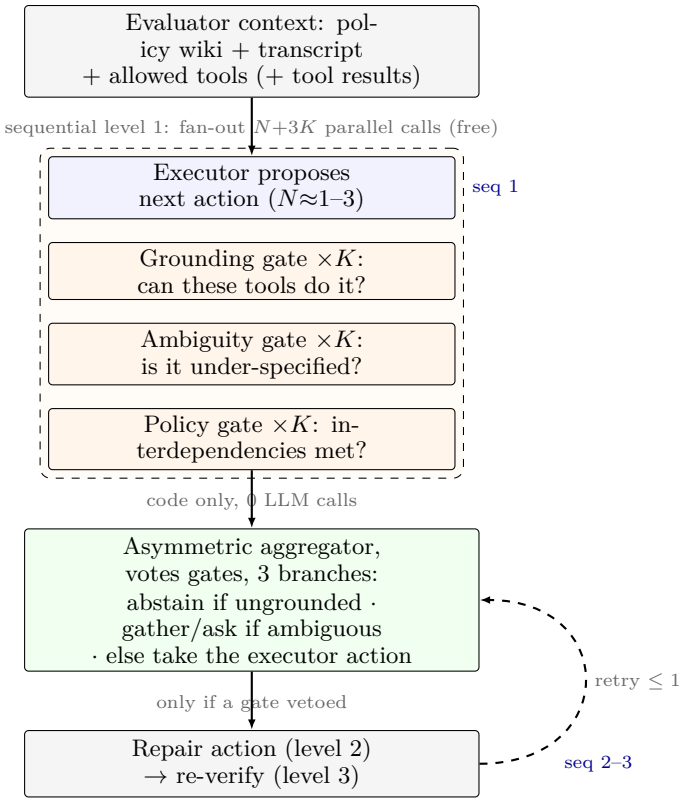

Figure 1: One decision step, annotated for the Track-2 audit. Parallel fan-out is $N+3K$ calls at sequential level 1 (dashed burst); the code aggregator adds no LLM call and branches three ways; a veto opens a retry loop (dashed) of at most one repair+re-verify. Deepest path = three sequential LLM calls (≤ 5).

is absent from the distribution. So the compute has to go somewhere the executor is not looking.

2 Architecture

Figure 1 shows one decision step and makes the four quantities the Track-2 rules ask for explicit: sequential LLM-call depth, parallel fan-out, branches, and the retry loop. At sequential level 1 the executor proposes an action while the three gates evaluate the same evaluator-provided context; all of these run concurrently, so the fan-out is $N+3K$ parallel calls but they count as a single sequential level. The aggregator is pure code (zero LLM calls) and branches three ways on the voted gate verdicts. Only a veto triggers the retry loop: one repair call (level 2) that re-verifies against the gates (level 3) and returns, capped at one retry. The deepest path is therefore three sequential LLM calls, against the limit of five, and the deterministic aggregator makes the decision rule itself perfectly Pass^3-consistent given a fixed set of gate votes.

3 Method

Grounding gate (Hallucination). A boolean "can this be done?" judgment does not move recall, because the error is systematic rather than variable. We instead decompose: the LLM enumerates the tools, parameters, and policy preconditions the goal requires, and deterministic code checks their presence against the allowed-tool set. The enumeration is voted $k\geq3$ within a trial, since gpt-oss is not bit-deterministic even at temperature 0 and the enumerated list is where the residual variance lives. We split the veto by evidence quality: a missing tool is a hard abstain (high precision, zero Base false positives across every sweep), while a missing parameter is only a soft flag, because the LLM routinely over-specifies a parameter that is not literally in the schema and would otherwise refuse valid Base requests.

Ambiguity gate and the disambiguation ladder. When the gate fires it does not immediately ask the user. It walks the paper's resolution ladder in priority order: strict policy, then explicit request, then learned preferences (which requires actually calling get_user_preferences), then heuristic defaults, then context (again a tool call), and asking only last. The key realisation is that preferences and context are not in the prompt, so the correct move is almost always an information-gathering tool call, not a clarifying question. The aggregator forces a clarifying question only when the gate votes are unanimous, which protects Base from the roughly one-in-three false-positive rate a single gate shows.

Policy gate (Base). A verifier for the conditional interdependencies in the wiki. For example, after a multi-stop route modification, policy LLM-POL:022 requires the agent to state that it took the fastest route and offer alternatives. This was the single most common systematic policy miss we found, and the target of our fourth lever.

Aggregator and effort. The aggregator is a fixed priority: grounding veto > ambiguity ladder > policy > executor action. Reasoning effort was tuned empirically and is counter-intuitive: the grounding gate is best at low effort (higher effort makes it over-think and refuse valid capabilities, and costs $\sim8\times$ the reasoning tokens), the ambiguity gate peaks at medium, and the executor runs at low. We ship greedy decoding (temperature 0) as the default after finding it fixes a large per-trial consistency gap.

4 Results

All headline numbers are on the complete public test split (125 tasks: 50 Base / 50 Hallucination / 25 Disambiguation), three trials, the shipped configuration; we report no hidden-test-set score or ranking, per the submission rules. Pass^3 is the strict metric; Pass@3 is the any-trial ceiling.

Table 1 and Table 2 give the outcome. For context, frontier single-pass models score (average Pass^3)

Split	Pass[1]	Pass[2]	Pass[3]	Pass@3
Base (50)	0.62	0.54	0.50	0.76
Hallucination (50)	0.62	0.50	0.42	0.78
Disambiguation (25)	0.52	0.36	0.32	0.64
Overall	0.587	0.467	0.413	0.727

Table 1: Pass^k on the public test split, shipped config.

	Base	Hall.	Disamb.	Avg
Raw gpt-oss-120b	0.36	0.35	0.11	0.27
Ours	0.50	0.42	0.32	0.413
Delta	+0.14	+0.07	+0.21	+0.14

Table 2: Pass^3 delta over the raw single-pass baseline. The largest gain is on Disambiguation, roughly a 3× relative lift.

Opus 4.6 0.58, GPT-5 0.54, ours 0.413, Gemini 2.5 Pro 0.38, Qwen3-32B 0.31, xLAM 0.16. On the two self-awareness splits we beat Gemini 2.5 Pro (Hall 0.42 vs 0.34, Disamb 0.32 vs 0.28); we trail GPT-5 and Opus on all three, which is a base-model gap rather than a scaffolding gap and not one we try to close.

Prove-or-kill: what actually worked. The honest story of this project is that most of what we built did not survive a matched ablation, and we report that as a feature. Table 3 summarises the four levers.

The ambiguity gate is the one genuine contribution. On sixteen unseen Disambiguation tasks the raw greedy scaffold scores a flat zero: it never disambiguates on its own, and the gate lifts that to a non-zero rate, reaching 0.32 on the full test split. This is the cleanest confirmation of the thesis: the gate does not select a better sample, it creates an information-gathering step that the executor's distribution does not contain. The inference-compute centrepiece is a single value-resolution case where greedy decoding deterministically locks the wrong choice (0/3), while sampling the executor at temperature 1 recovers the correct value in 7 of 8 draws, and the gate is what decides when to pay for that sampling.

The grounding gate did not generalise. Its early "win" was overfit to the four tasks we diagnosed it on; on unseen missing-tool tasks the gated and un-gated arms are identical (0.625 either way). The reason is structural: when the removed tool has an available substitute, an availability check cannot veto a wrong-but-available action, and both arms fabricate the same workaround. The Hallucination lift over the baseline (0.35 → 0.42) is the scaffold and greedy decoding, not the gate, and we say so rather than claim it.

The same is true of Base: the +0.14 (0.36 → 0.50) is the planner/executor scaffold and greedy decoding, with no gate contributing. On Base the gates are deliberately near-inert: the grounding gate produces zero over-refusals across every trial, and the ambiguity ladder exits immediately when nothing is under-specified, so they protect the split rather than lift it. The one module that did act on Base, the policy gate, hurt it.

Lever	Matched-ablation outcome	Verdict
Ambiguity gate	unseen disamb 0.000 → 0.125; test 0.32	keep
Grounding gate	unseen missing-tool Hall 0.625 = 0.625	inert
Policy gate	train Base 0.286 → 0.095	killed
Over-action rule	held-out disamb 0.125 → 0.000	reverted

Table 3: Every lever measured against its own control. Only the ambiguity gate produces behaviour the baseline lacks; the rest are boundaries.

The policy gate was net-negative and we removed it. A matched train ablation dropped Base Pass^3 from 0.286 to 0.095: the repairer rewrote respond turns on tasks the scaffold already handled and regressed them (base_66 went 3/3 to 0/3) while the intended gains were tiny. It ships default-off. Likewise a blanket "do only what was asked" instruction backfired: on Disambiguation the correct behaviour is extra information-gathering, so the rule suppressed exactly what the ambiguity gate induces, and we reverted it.

Compute and latency. Two assumptions are needed to read these numbers. First, the latency we report is agent LLM time only; the end-to-end wall clock is dominated by the Gemini user-simulator and judge, which are part of the evaluator, not our agent, so we exclude them. Second, "per step" means one decision step, in which the executor and all gates issue their calls concurrently. Under those assumptions the median decision step is ∼2.5 s (Base/Disamb 2.1 s, Hall 3.6 s; p90 6.4 s) and total agent LLM time is ∼12.6 s per task. Against the paper's GPT-5 at ∼22 s per step, we are roughly 9× faster per step while doing more per step, because the ensemble fires in one parallel burst. On tokens, the executor-plus-gates burst runs about eight parallel calls of ∼3–4k tokens each, so a ∼10-step task consumes on the order of 300k tokens, inside the 500k input+reasoning+output average budget, with headroom to escalate hard steps. This is the Track-2 argument: multi-pass reliability at a latency an in-car assistant could actually deploy.

Honesty caveats. Three, stated before anyone else can. (i) The raw baseline was scored on the 254-task train+test set that includes the 129 tasks we tuned on, whereas our 0.413 is the 125-task held-out split; the submission is matched on the hidden set by construction, but the two numbers here are not the same task set. (ii) A tightly-tuned run once showed Base at 1.0, but that was a small easy subset; honest Base on the full split is 0.50. (iii) Disambiguation 0.32 is higher than a 0.125 held-out slice we also ran (different small samples), so we anchor the claim on the raw≈0 baseline, which no sampling can move, rather than on the friendlier of two numbers.

5 Compliance

Our harness performs self-verification against evaluator-provided inputs only: the system prompt, policy wiki, transcript, tool definitions, and tool results, which the competition rules (§3) list as allowed: "multi-pass reasoning, private planning, self-verification, retries, ensembles, and parallel calls." It does not self-evaluate against task-level metrics, which §4 prohibits ("checking subscores and re-prompting the LLM … iteratively repair[ing] against the scoring criteria"). We cannot do the prohibited thing: the A2A contract never returns subscores, reward, or answer keys to the agent, and the gate and aggregator code import no benchmark reward or evaluator module (zero references to reward, r_actions, r_policy, or answer_key). A gate reasons about tool availability and policy preconditions drawn from the provided wiki, not about whether a score would be 1. The abstention "I cannot do X because tool Y is missing" is the genuine response, and the repair loop re-verifies against our own gate verdict, never a scoring signal. All tuning is on the 129 training tasks via overall pass/fail only, never per-subscore, never on the hidden set.

6 Conclusion

CAR-bench's self-awareness failures are unanimous, not noisy, so the natural best-of-N use of Track-2 compute is inert. The compute is better spent manufacturing the dissent the executor never produces on its own: independent verifier gates, voted for stability, aggregated by code, vetoing asymmetrically. The disciplined result is one gate that genuinely works, three levers we killed with matched ablations, and a +0.14 average Pass^3 delta at deployable latency. The negative results are not incidental; they are the evidence that the surviving contribution is real.

References

[Cerebras Systems, 2024] Cerebras Systems. Cerebras inference: Fast LLM inference on the wafer-scale engine. https://www.cerebras.ai/inference, 2024.

[Google, 2025] Google. Agent2agent (A2A) protocol. https://a2a-protocol.org/, 2025.

[Kirmayr et al., 2026] Johannes Kirmayr, Lukas Stappen, and Elisabeth Andre. CAR-bench: Evaluating the consistency and limit-awareness of LLM agents under real-world uncertainty. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens, editors, Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pages 40599–40618, San Diego, California, United States, July 2026. Association for Computational Linguistics.

[Lu et al., 2025] Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Felix Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. ToolSandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities. arXiv preprint arXiv:2408.04682, 2025.

[OpenAI, 2025] OpenAI. gpt-oss-120b & gpt-oss-20b model card. https://openai.com/index/gpt-oss-model-card/, 2025.

[Yao et al., 2024] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. arXiv preprint arXiv:2406.12045, 2024.